

Aula 14 - PHPUnit em testes

Descrição: Usar o PHPUnit para cobrir toda a pirâmide de testes é uma excelente estratégia para manter a saúde do seu código. Nesta aula vamos consolidar os conceitos aprendidos em aula utilizando o framework phpunit para realizar testes.

1. Introdução

Embora o PHPUnit tenha sido desenhado nativamente para testes unitários, a sua robustez e o seu ecossistema permitem que ele funcione como o "maestro" de testes de integração e até E2E (End-to-End), desde que pareado com as ferramentas certas. Vamos entender como estruturar e utilizar o PHPUnit partindo do zero absoluto no Windows.

2. Configuração Inicial no Windows

Configurar o PHPUnit no Windows pode parecer intimidador por conta das variáveis de ambiente e caminhos de arquivos (\), mas seguindo a ordem certa, é um processo bem direto. Abra o seu terminal (PowerShell ou CMD).

Pré-requisitos (O básico essencial)

Antes de começar, você precisa ter certeza de que o PHP e o Composer estão instalados globalmente no seu Windows. Digite:

Bash

```
php -v  
composer -v
```

Nota: Se ambos retornarem as versões instaladas, você está pronto. Se o Windows disser que o comando não foi reconhecido, você precisará adicionar o PHP e o Composer às suas Variáveis de Ambiente (PATH) do Windows antes de continuar.

Passo 1: Criar a Pasta do Projeto

Vamos criar uma pasta limpa para o seu projeto e entrar nela:

Bash

```
mkdir meu-projeto-testes  
cd meu-projeto-testes
```

Passo 2: Inicializar o Composer e Instalar o PHPUnit

O PHPUnit deve ser instalado como uma dependência de desenvolvimento do seu projeto. Isso garante que ele não vá para o servidor de produção desnecessariamente. Execute o comando:

Bash

```
composer require --dev phpunit/phpunit
```

Passo 3: Configurar o Autoloading no composer.json

Para que o PHPUnit encontre suas classes de código e suas classes de teste automaticamente, precisamos configurar o padrão PSR-4. Abra o arquivo composer.json gerado na raiz e adicione as seções "autoload" e "autoload-dev":

JSON

```
{  
  "require-dev": {  
    "phpunit/phpunit": "^11.0"  
  },  
  "autoload": {  
    "psr-4": {
```

```

        "App\\": "src/"
    }
},
"autoload-dev": {
    "psr-4": {
        "Tests\\": "tests/"
    }
}
}
}

```

Após salvar o arquivo, vá ao terminal e atualize o mapa de classes do Composer:

Bash

```
composer dump-autoload
```

Passo 4: Criar a Estrutura de Pastas

Crie manualmente (aproveitando o terminal do Windows) duas pastas na raiz do projeto:

Bash

```
mkdir src
mkdir tests
```

- src/: Onde ficará o código da sua aplicação.
- tests/: Onde ficarão os seus testes.

Passo 5: Criar o Arquivo de Configuração Base (phpunit.xml)

O PHPUnit precisa de um arquivo de instruções para saber onde procurar os testes. Crie o arquivo phpunit.xml na raiz do projeto:

XML

```

<?xml version="1.0" encoding="UTF-8"?>
<phpunit xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:noNamespaceSchemaLocation="https://schema.phpunit.de/11.0/phpunit.xsd"
    bootstrap="vendor/autoload.php"
    colors="true">
    <testsuites>
        <testsuite name="Testes do Projeto">
            <directory>tests</directory>
        </testsuite>
    </testsuites>
</phpunit>

```

3. Escrevendo o Código e o Primeiro Teste Real

Vamos criar uma classe simples de calcular e, em seguida, testá-la para validar o ambiente.

O Código da Aplicação

Dentro da pasta src/, crie o arquivo Calculadora.php:

PHP

```

<?php

namespace App;

class Calculadora
{

```

```
public function somar(float $a, float $b): float
{
    return $a + $b;
}
}
```

O Código do Teste

Dentro da pasta tests/, crie o arquivo CalculadoraTest.php. (Lembre-se: por padrão, o PHPUnit exige que o arquivo e a classe terminem com a palavra Test).

PHP

```
<?php
```

```
namespace Tests;
```

```
use PHPUnit\Framework\TestCase;
```

```
use App\Calculadora;
```

```
class CalculadoraTest extends TestCase
```

```
{
    public function testDeveSomarDoisNumerosCorretamente()
    {
        // 1. Cenário (Arrange)
        $calculadora = new Calculadora();

        // 2. Ação (Act)
        $resultado = $calculadora->somar(10, 5);

        // 3. Validação (Assert)
        $this->assertEquals(15, $resultado);
    }
}
```

Rodando os Testes no Windows

No Windows, os executáveis locais do Composer ficam dentro da pasta vendor\bin\. Execute o seguinte comando na raiz do seu projeto usando o PowerShell ou CMD:

Bash

```
.\vendor\bin\phpunit
```

Se tudo deu certo, você verá a famosa resposta com a barra verde de sucesso:

Plaintext

PHPUnit 11.x.x by Sebastian Bergmann and contributors.

Runtime: PHP 8.x.x

Configuration: C:\meu-projeto-testes\phpunit.xml

. 1 / 1 (100%)

Time: 00:00.012, Memory: 6.00 MB

OK (1 test, 1 assertion)

O ponto . significa que um teste passou com sucesso. Se o teste falhasse, você veria um F vermelho e o motivo da falha.

4. Turbinando a Produtividade: Integração com o VS Code

Integrar o PHPUnit diretamente no VS Code transforma completamente a experiência de desenvolvimento no Windows, eliminando a necessidade de digitar comandos no terminal a todo momento.

Passo 1: Instalar as Extensões Certas

Vá em Extensões (Ctrl + Shift + X), procure e instale:

1. PHPUnit Test Explorer (de Holger Benl): Integra os testes à aba nativa de testes do VS Code (ícone do béquer/fita de laboratório).
2. Better PHPUnit (de Caleb Porzio): Ideal para rodar testes rapidamente usando atalhos de teclado.

Passo 2: Configurar os Caminhos no Windows

Na raiz do seu projeto, crie uma pasta chamada .vscode e, dentro dela, o arquivo settings.json , colando as seguintes diretrizes para o Windows:

JSON

```
{
  "phpunit.phpunit": "vendor/bin/phpunit",
  "phpunit.xml": "phpunit.xml",
  "better-phpunit.phpunitBinary": "vendor/bin/phpunit",
  "better-phpunit.commandSuffix": "--colors=always"
}
```

Passo 3: Utilizando a Interface Visual e Atalhos

- **Pelo Test Explorer:** Clique no ícone do Béquer na barra lateral. O VS Code listará seus testes automaticamente. Basta clicar no botão de Play (▶). Ao abrir o arquivo CalculadoraTest.php, você também verá um botão de Play acima do método.
- **Pelos Atalhos de Teclado (Better PHPUnit):** Com o cursor posicionado dentro do método de teste, abra a paleta de comandos (Ctrl + Shift + P) e digite Better PHPUnit: run. (Dica: Você pode mapear este comando para o atalho Ctrl + K, Ctrl + T).

Dica de Ouro (Autorun): Instale a extensão Run on Save e configure-a para disparar o comando .\vendor\bin\phpunit sempre que um arquivo .php for salvo.

5. Aprofundando a Utilidade: Dominando a Pirâmide de Testes

Agora que seu ambiente está configurado e integrado, podemos explorar o verdadeiro poder e versatilidade do PHPUnit atuando nos três níveis da pirâmide.

A) Testes Unitários Avançados (Isolamento com Mocks)

O objetivo do teste unitário é testar a menor parte do código de forma isolada. Se a classe depende de regras complexas ou outras classes, nós isolamos essas dependências usando Mocks.

PHP

```
use PHPUnit\Framework\TestCase;
```

```
class CalculadoraDescontoTest extends TestCase
{
    public function testDeveCalcularDescontoParaClienteVIP()
    {
        // Criando um Mock (dublê) da dependência do Usuário
        $usuarioMock = $this->createMock(Usuario::class);
```

```

$usuarioMock->method('isVIP')->willReturn(true);

$calculadora = new CalculadoraDesconto();
$resultado = $calculadora->calcular(100, $usuarioMock);

$this->assertEquals(80, $resultado); // 20% de desconto
}
}

```

- Características: Executam em milissegundos e não tocam em recursos externos (banco, rede, arquivos).

B) Testes de Integração (Componentes trabalhando juntos)

Aqui queremos que o PHPUnit converse com recursos reais, como salvar arquivos ou interagir com o banco de dados. Geralmente usamos bancos em memória para garantir velocidade e limpeza entre as execuções.

PHP

```
use PHPUnit\Framework\TestCase;
```

```

class UsuarioRepositoryTest extends TestCase
{
    private PDO $db;

    protected function setUp(): void
    {
        // Configura uma conexão SQLite em memória antes de CADA teste
        $this->db = new PDO('sqlite::memory:');
        $this->db->exec("CREATE TABLE usuarios (id INTEGER PRIMARY KEY, nome TEXT)");
    }

    public function testDeveInserirUsuarioNoBancoDeDados()
    {
        $repository = new UsuarioRepository($this->db);
        $repository->salvar(new Usuario('Rodrigo'));

        // Valida diretamente no banco se o registro foi persistido
        $stmt = $this->db->query("SELECT COUNT(*) FROM usuarios WHERE nome = 'Rodrigo'");
        $this->assertEquals(1, $stmt->fetchColumn());
    }
}

```

C) Testes de Ponta a Ponta (E2E / Funcionais)

O PHPUnit puro não simula ações em um navegador de internet. Para testes E2E, integramos o PHPUnit a ferramentas de automação de browser. No ecossistema PHP, as principais são o Symfony Panther , o Laravel Dusk e o Mink.

Exemplo utilizando o Symfony Panther rodando sob o motor do PHPUnit:

PHP

```
use Symfony\Component\Panther\PantherTestCase;
```

```

class LoginE2ETest extends PantherTestCase

```

```

{
    public function testUsuarioConsegueFazerLogin()
    {
        // Abre um navegador real (ou em modo headless)
        $client = static::createPantherClient();
        $crawler = $client->request('GET', '/login');

        // Interage com os elementos da tela do Windows
        $form = $crawler->selectButton('Entrar')->form([
            '_username' => 'admin@email.com',
            '_password' => '123456',
        ]);
        $client->submit($form);

        // Realiza a asserção do PHPUnit no resultado final renderizado
        $this->assertSelectorTextContains('.dashboard-title', 'Bem-vindo, Admin!');
    }
}

```

Organizando a Arquitetura no phpunit.xml

Para que os testes de integração e E2E (que são naturalmente mais lentos) não atrasem o seu desenvolvimento local, separamos as suítes no phpunit.xml:

XML

```

<?xml version="1.0" encoding="UTF-8"?>
<phpunit bootstrap="vendor/autoload.php" colors="true">
    <testsuites>
        <testsuite name="Unitario">
            <directory>tests/Unit</directory>
        </testsuite>
        <testsuite name="Integracao">
            <directory>tests/Integration</directory>
        </testsuite>
        <testsuite name="E2E">
            <directory>tests/E2E</directory>
        </testsuite>
    </testsuites>
</phpunit>

```

Dessa forma, para rodar apenas os testes rápidos de unidade no seu terminal do Windows, basta executar:

Bash

```

.\vendor\bin\phpunit --testsuite Unitario

```